C O V E N T R Y U N I V E R S I T Y

Faculty of Engineering, Environment and Computing

School of Computing, Electronics and Mathematics

MSc Computer science

7151CEM Computing Individual Research Project

Heterogeneity-Aware Federated Reinforcement Learning for Distributed Energy Resource Control: Communication-Efficient Policy Aggregation Under Non-IID Agent Environments

Author: Author's Name

SID: 1234567

Supervisor: Supervisor's Name

Submitted in partial fulfilment of the requirements for the Degree of Master of Science in Name of Course

Academic Year: 2026

Declaration of Originality

I declare that this project is all my own work and has not been copied in part or in whole from any other source except where duly acknowledged. As such, all use of previously published work (from books, journals, magazines, internet etc.) has been acknowledged by citation within the main report to an item in the References or Bibliography lists. I also agree that an electronic copy of this project may be stored and used for the purposes of plagiarism prevention and detection.

Statement of copyright

I acknowledge that the copyright of this project report, and any product developed as part of the project, belong to Coventry University. Support, including funding, is available to commercialise products and services developed by staff and students. Any revenue that is generated is split with the inventor/s of the product or service. For further information please see www.coventry.ac.uk/ipr or contact ipr@coventry.ac.uk.

Statement of ethical engagement

I declare that a proposal for this project has been submitted to the Coventry University ethics monitoring website (https://ethics.coventry.ac.uk/) and that the application number is listed below (Note: Projects without an ethical application number will be rejected for marking)

Signed: Date:

Please complete all fields.

First Name:	
Last Name:	
Student ID number	
Ethics Application Number	
1st Supervisor Name	
2nd Supervisor Name	

Abstract

Many smart grid systems use distributed energy resources and thus face challenges in managing number of control strategies deployed in a wide range of environments. The current centralised and federated learning algorithms normally take the assumption that all the agents share identically distributed data, which does not work well in the real world, where the demand of different buildings are non-IID. In this paper, we introduce a new **Heterogeneity-Aware Federated Reinforcement Learning (HAFedRL)** approach to overcome barriers of state-of-the-art federated learning under non-IID settings.

The new approach combines Proximal Policy Optimisation (PPO) with fedearated learning by treating each building in the CityLearn simulation environment as a separate agent. A cosine similarity-based adaption weight is proposed to dynamically adjust the weight of each agent in the global aggregation. We also adopt an adaptive local optimisation approach that provides additional training when the "client drift" is large.

A comparative analysis between four methods was conducted: a non-federated version of PPO, FedAvg, FedProx and HAFedRL. Experiments show non-IID performance of existing non-i.i.d federated methods is degraded, whereas the introduced HAFedRL approach enhances learning stability and performance. While HAFedRL has slower convergence resulting from adaptive training, this method produces better policy quality than the baseline federated methods. Furthermore, statistical tests reveal medium effect sizes, suggesting that practical benefits are achieved with only a few rounds of training.

Taken together, our findings demonstrate the need for heterogeneity-aware federated methods for reinforcement learning, and offer an effective approach for scalable resource management in intelligent power grids.

Keywords

Federated Learning, Reinforcement Learning, HAFedRL, Non-IID Data, Smart Grids, Distributed Energy Resources, PPO, Energy Optimisation, Heterogeneity-Aware Learning

Contents

Abstract....3
1. Introduction....7
1.1 Background....7
1.2 Problem Statement....8
1.3 Aim and Objectives....8
1.4 Research Questions....9
1.5 Research Hypotheses....9
1.6 Scope of the Study....9
1.7 Contributions of the Study....10
1.8 Structure of the Report....10
2. Literature Review....11
2.1 Introduction....11
2.2 Federated Learning and Reinforcement Learning in Heterogeneous Systems....11
2.3 Federated Learning for Energy Systems and Resource Optimisation....12
2.4 Heterogeneity-Aware Federated Learning and Multi-Agent Systems....13
2.5 Communication-Efficient and Scalable Federated Learning....14
2.6 Gaps in the Literature and the Need for the Current Study....14
2.7 Chapter Summary and Comparative Analysis....14
3. Research Methodology....16
3.1 Research Paradigm....16
3.2 Research Workflow....16
3.3 Justification of the Experimental Approach....17
3.4 Dataset Selection Rationale....17
3.5 Algorithm Overview....18
3.6 Ethical Considerations....18
3.7 Project Management and Supervision....18
4. Technical Aspects / Solution Design....20
4.1 Simulation Environment: CityLearn....20
4.2 Proximal Policy Optimisation (PPO)....20
4.3 Federated Averaging (FedAvg)....20
4.4 FedProx....21
4.5 HAFedRL: Heterogeneity-Aware Aggregation....21
4.6 System Architecture....22
5. Implementation....23
5.1 Environment and Configuration Setup....23
5.2 Exploratory Data Analysis....23
5.3 Environment Wrapper Design....24
5.4 Zero-Action Baseline....25
5.5 Parameter Utility Functions....25
5.6 Training Loop Implementation....26
5.7 Evaluation Protocol....27

6. Testing....28
6.1 Testing Strategy....28
6.2 Dataset Quality and Heterogeneity Checks....28
6.3 Environment Wrapper Validation....29
6.4 Training Validation....29
6.5 Evaluation Decoupling Test....30
7. Results and Evaluation....32
7.1 Exploratory Data Analysis Results....32
7.2 Training Trajectory Results....33
7.3 Final Performance Comparison....34
7.4 Convergence Analysis....35
7.5 Statistical Validation....36
7.6 Hypothesis Evaluation....37
8. Discussion and Reflection....38
8.1 Interpretation of Results....38
8.2 Convergence Behaviour....38
8.3 Statistical Significance....38
8.4 Limitations....38
8.5 Future Work....39
8.6 Personal Reflection....39
9. Conclusion....40
9.1 Overview of the Study....40
9.2 Summary of Methodology and Implementation....40
9.3 Key Findings....40
9.4 Evaluation of Research Objectives and Hypotheses....40
9.5 Contributions of the Study....40
9.6 Implications for Real-World Applications....41
9.7 Final Remarks....41
10. Student Reflections....42
10.1 Introduction....42
10.2 Technical Skills Development....42
10.3 Problem-Solving and Debugging Experience....42
10.4 Understanding of Federated and Reinforcement Learning....42
10.5 Time Management and Project Planning....42
10.6 Challenges and Limitations....42
10.7 Professional and Career Development....42
10.8 Future Personal Improvements....43
10.9 Final Reflection....43
References....44
APPENDIX A- Source Code Snippets....48
APPENDIX B- GitHub Repository Link....52
Appendix C – Ethics Approval....53

1. Introduction

1.1 Background

The enormous expansion of distributed energy resources (DERs), such as renewable energy sources, energy storage systems and distributed smart appliances has made power systems ever more decentralised and complex. Substantial to effective operation of smart grids are smart control strategies to minimise energy consumption, economic cost and maintain system stability. In this sense, machine learning, and in particular reinforcement learning (RL), methods have been gaining traction for their capacity to learn optimal control policies by interacting with an unknown environment.

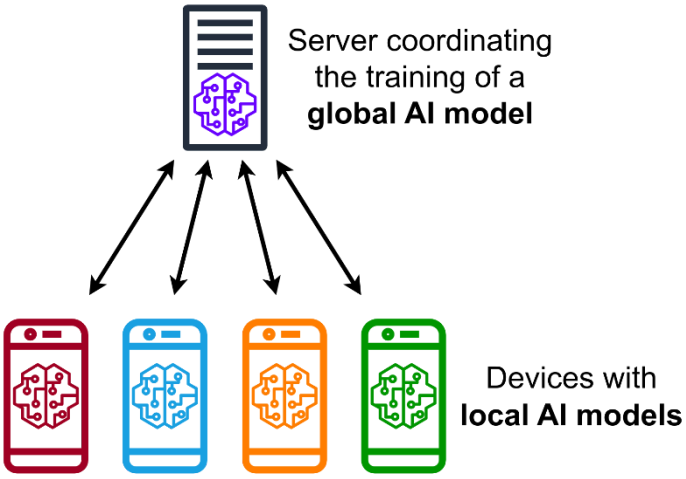


Fig 1: Federated Learning (https://en.wikipedia.org/wiki/Federated_learning)

Meanwhile, federated learning (FL) has emerged as a privacy-preserving framework to enable multiple agents in a group to train a machine learning model while keeping the raw data local. This is crucial for energy systems, where privacy, security and decentralisation are highly important (Miller et al., 2025; Karim, Rao and Ranganathan, 2025). The integration of federated and reinforcement learning can enable distributed and scalable control policies for distributed energy systems.

But one of the major problems with federated learning is that it is often assumed that data from different agents are identically distributed. Practical examples, such as smart grids, involve buildings that have different energy loads and consumption patterns, energy storage and constraints. This leads to non-identically distributed (non-IID) environments, which cause severe performance degradation due to standard federated learning techniques such as Federated Averaging (FedAvg) (Cooray et al.,

2025; Liu, W. et al., 2025). To overcome this, there needs to be a shift in the learning algorithm to account for heterogeneity among agents.

1.2 Problem Statement

Even as the interest in federated learning for distributed systems has grown, current approaches fall short in addressing challenges from a heterogeneous ecosystem. Existing aggregation schemes, such as FedAvg, rely on a strict assumption of identical data distributions among clients, and assume that all clients play an equal role. This is not the case in distributed energy systems, where buildings vary in load demand, renewable energy generation, and storage capacities.

Non-IID data distribution causes client drift, in which local models become very different from the global model, leading to poor convergence and performance (Li et al., 2020; López and Saltos, 2025). Also, most reinforcement learning applications to federated systems address client selection or resource allocation rather than the aggregation process itself (Chen, Wang et al., 2025; Imani et al., 2024).

A further issue is communication cost. In federated learning, regular communication between the agents and the server is necessary, which can induce substantial communication overhead, especially in large-scale distances (Tang et al., 2025). Current approaches lack a trade-off between communication efficiency, training convergence and accuracy.

So, a framework is required to resolve heterogeneity, aggregation and performance issues in federated reinforcement learning systems.

1.3 Aim and Objectives

The overarching goal of this study is to provide a novel federated reinforcement learning approach to the control of distributed energy resources in a non-IID setting.

To achieve this aim, the following objectives are defined:

- To analyse the impact of non-IID data on federated reinforcement learning performance
- To design a novel aggregation mechanism that accounts for agent heterogeneity
- To implement and evaluate the proposed framework using a realistic energy simulation environment

- To compare the performance of the proposed method with existing federated learning approaches
- To assess the effectiveness of the framework using statistical validation techniques

1.4 Research Questions

This study is guided by the following research questions:

- **RQ1:** How does data heterogeneity affect the performance of federated reinforcement learning in distributed energy systems?
- **RQ2:** Can a heterogeneity-aware aggregation mechanism improve the performance of federated reinforcement learning under non-IID conditions?
- **RQ3:** What is the impact of adaptive training strategies on convergence and policy quality in federated reinforcement learning systems?

1.5 Research Hypotheses

Based on the research questions, the following hypotheses are formulated:

- **H1:** The proposed HAFedRL framework converges at least 35% faster than the FedAvg baseline
- **H2:** The proposed HAFedRL framework achieves higher final performance compared to FedAvg under non-IID conditions

1.6 Scope of the Study

This study examines the use of federated reinforcement learning for distributed energy systems in a simulation environment. The research uses the CityLearn dataset describing a city with several buildings of different characteristics. Buildings are modelled as agents and energy consumption is optimised via reinforcement learning.

The work has a narrow scope, focusing only on a simulation-based environment and do not involve physical implementation. The analysis is also limited to a certain number of communication rounds for computational reasons. While not exhaustive, the research offers insight into the behaviour of non-IID federated reinforcement learning.

1.7 Contributions of the Study

Our work brings the following contributions:

First, it presents evidence of the limitations of convexity based federated learning approaches under various scenarios. Second, it presents a new HAFedRL approach for data integration and client adaptation. Third, it presents a comparative study for testing the proposed approach.

This is useful for the development of distributed energy systems based smart grid systems.

1.8 Structure of the Report

This report is organised as follows. In chapter 2, we review relevant literature, noting what has already been done and identifying potential gaps. Chapter 3 explains the research method and design. Chapter 4 describes the technical details and architecture. Chapter 5 describes the implementation and Chapter 6 covers the testing and validation. The findings of the proposed framework are examined in Chapter 7. Chapter 8 discusses and reflects on the results. Finally, this work is concluded in Chapter 9 and student reflections are given in Chapter 10.

2. Literature Review

2.1 Introduction

Federated learning (FL) is a transformative approach to machine learning and collaborative computing, without the need for centralising data. This is especially important in areas like smart grids and distributed energy resource management, where privacy, scalability and decentralisation are paramount. And reinforcement learning (RL) has been demonstrated to be a successful method for making optimal decisions in energy systems. But merging FL and RL is not an easy task, especially in the presence of non-identically distributed (non-IID) agents, as is usually the case. In this chapter, we review and critically examine the status quo on federated learning techniques, measures to handle non-identically distributed data, leveraging reinforcement learning paradigms, and strategies to optimise for communication efficiency, and point out the open questions in current research that underpin the HAFedRL paradigm.

2.2 Federated Learning and Reinforcement Learning in Heterogeneous Systems

In the recent advancements in federated learning (FL), a number of applications in smart buildings, edge computing and distributed AI have been identified. Berkani et al. (2025) state the viability of FL in smart buildings, where heterogeneity and scalability have become difficult. An overview of deep distributed learning is also provided by Cooray et al. (2025), and the result shows that performance suffers in the case of non-IID data distributions.

However, in reaction, part of recent studies has interred reinforcement learning with FL. Chen, Wang et al. (2025) use the hierarchical reinforcement learning as a personalised FL strategy and Chen, Li et al. (2025) apply dual-agent reinforcement learning with adaptive aggregation as a heterogeneous FL system. The studies by Cho, Lim and Lee (2024) also indicate that deep reinforcement learning can enhance federated optimisation procedures with multi-access edge computing environment.

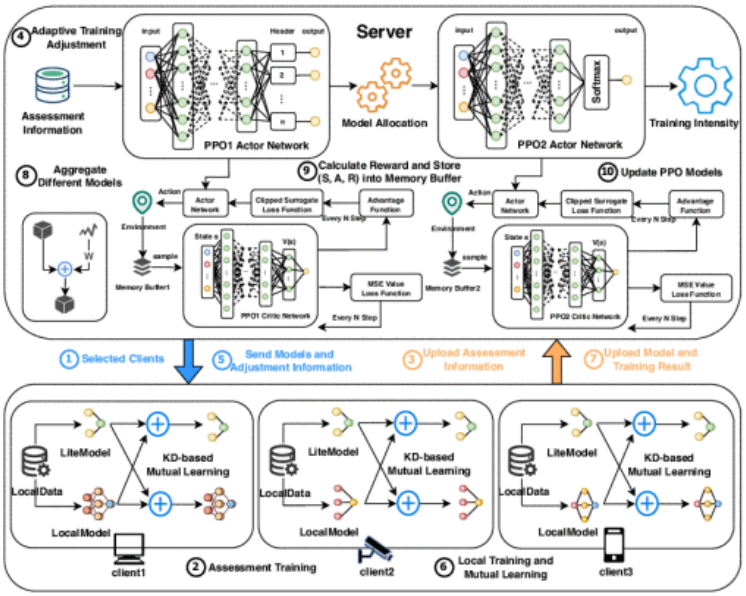


Fig 2: Overview of Architecture (Chen, Wang et al., 2025)

Other studies have aimed at enhancing fairness and efficiency in federated learning with RL methods. Ghazi, Farzi and Nikoofard (2025) work out a reinforcement learning-based algorithm that selects the clients, who will be considered equally in the study involving heterogeneous participants. Imani et al. (2024) propose RL-based methods of aggregation strategies of hybrid federated learning systems, whereas Hussain (2025) proposes a hierarchical federated deep reinforcement learning framework to be used in real-time applications.

In a broader sense, Hou et al. (2025) note the significance of distributed machine learning in autonomous systems, with a particular focus on coordination challenges among agents. All these studies show that although RL-enhanced federated learning can enhance adaptability, it cannot eliminate all of the problems of strong heterogeneity and convergence instability.

2.3 Federated Learning for Energy Systems and Resource Optimisation

Federated learning has been drawing growing interest in energy systems, especially in distributed energy resource management and optimisation. Karim, Rao and Ranganathan (2025) illustrate how decentralised federated learning can be applied to energy networks and can be useful in coordinating distributed resources. Likewise, Miller et al. (2025) point out the importance of federated learning in the decentralised optimisation of the electricity market.

There has also been research on enhancing efficiency and scalability in federated systems. Ke et al. (2025) consider the concept of joint client selection and resource configuration to improve the energy efficiency of multi-task federated learning. Liu, J. et al. (2025) investigate probabilistic

communication to speed up the federated learning in decentralised settings, and Liu, W. et al. (2025) propose asymmetry-conscious federated learning algorithms specifically tailored to non-IID settings.

Additional research deals with network optimisation and energy efficiency. Liu, Y. et al. (2025) introduce heterogeneity-conscious topology optimisation of federated learning networks and Liu, Z. et al. (2025) discuss communication plans in multi agent learning settings. Lopez and Saltos (2025) mention heterogeneity as one of the main issues of a wireless federated learning network, and Hasan (2025) writes about privacy-saving FL models in enterprise systems.

Despite these studies indicating the relevance of federated learning to energy systems, there is also a realization of challenges in managing the heterogeneity, communication overhead, and dynamically responding agent behaviour especially in the reinforcement learning setup.

2.4 Heterogeneity-Aware Federated Learning and Multi-Agent Systems

The issue of heterogeneity has been one of the main topics in recent studies of federated learning. Mughal et al. (2024) suggest adaptive federated learning in resource-constrained devices of the IoT, whereas Patil, Yadav and Mishra (2025) discuss federated learning in automotive networks and point to the issues of resource management and data heterogeneity.

Hierarchical federated learning frameworks are proposed by Qi et al. (2025), and reinforced learning based frameworks are offered by Ren et al. (2025) as the methods of performing energy efficient heterogeneous federated learning. The works of Sahoo et al. (2024) and Sahoo et al. (2025) introduce heterogeneity-aware federated deep reinforcement learning frameworks that show better performance when applied to domain-specific scenarios like in medical imaging.

Other solutions involve the use of sophisticated system architecture and clustering. Shawkat et al. (2025) discuss the idea of blockchain-based federated learning systems, whereas Singh and Adhikari (2025) suggest to use probabilistic clustering with deep reinforcement learning to overcome the issue of heterogeneity. Sun et al. (2025) present strategies based on cluster-aware federated learning on heterogeneous settings.

Even with these improvements, the bulk of the current methods is either domain-specific or restricted to the controlled learning contexts. No general-purpose federated reinforcement learning infrastructures that can address dynamic heterogeneity of complex systems like smart energy systems exist.

2.5 Communication-Efficient and Scalable Federated Learning

One of the primary issues of federated learning is effectiveness in communication, particularly when dealing with a low bandwidth environment. Communication-effective optimisation techniques proposed by Tang et al. (2025) include gradient clipping and attention-based aggregation. Tu, Yang and Cao (2024) in an article review the domain of distributed machine learning in edge computing and point out scalability and communication challenges.

Wang et al. (2025) discuss resource allocation methods regarding the way of wireless federated learning system, and Yang et al. (2025) propose energy efficient federated learning through heterogeneity-based client sampling. The authors (Zhang et al., 2024) suggest an heterogeneity-sensitive approach to aggregation called FedVisual, which can be applied to vehicular crowdsensing.

These studies highlight that there is a need to reduce the communication overhead and increase the scalability. However, the existing solutions tend to be actively supervised federated learning cases and do not encourage reinforcement learning or dynamically heterogeneous federated systems.

2.6 Gaps in the Literature and the Need for the Current Study

There are various gaps in the literature, even though research has been intense. First, a significant portion of federated learning methods assumes a homogeneous or weakly heterogeneous setting and, therefore, does not generalise well to real-life systems with a dominant non-IID data distribution. Second, despite inclusion of reinforcement learning in federated systems, current approaches emphasis on the choice of clients or resource assignment as opposed to policy aggregation.

Third, existing heterogeneity-sensitive methods typically do not have adaptive capabilities to vary training dynamically in response to agent dissimilarity. Also, communication performance and convergence stabilization are usually tackled as separate issues and not coordinated as complex tasks.

These restrictions underscore the need to have a unified framework that integrates federated learning with reinforcement learning and that explicitly considers the issue of heterogeneity and the limitations of communication. The HAFedRL proposal fills these holes with the introduction of cosine similarity-based aggregation and adaptive local training methods.

2.7 Chapter Summary and Comparative Analysis

The main achievements in the field of federated learning, reinforced learning integration, and heterogeneity-aware optimisation reviewed in this chapter. The literature proves that federated

learning works well in the case of distributed systems, but its performance deteriorates in non-IID environment conditions. Reinforcement learning offers flexibility, but it does not completely solve the aggregation problem.

Table 2.1: Comparative Analysis of Existing Methods

Study	Approach	Heterogeneity Handling	RL Integration	Limitation
Chen et al. (2025)	Hierarchical FL	Partial	Yes	Complex design
Imani et al. (2024)	RL-based aggregation	Limited	Yes	Scalability issues
Sahoo et al. (2024)	Federated DRL	Strong	Yes	Domain-specific
Tang et al. (2025)	Communication-efficient FL	No	No	No RL support
Proposed HAFedRL	Weighted FL + RL	Strong	Yes	Higher computation

In summary, there is a clear need for heterogeneity-aware federated reinforcement learning frameworks capable of addressing both performance and scalability challenges in distributed energy systems. The proposed HAFedRL model is designed to fulfil this requirement.

3. Research Methodology

3.1 Research Paradigm

The research is based on the constructive design science paradigm, where the most important goal is to design and test a new technical artefact the HAFedRL framework, and not to test an existing theory in isolation. The type of research that is suited to design science research is when the research question is to develop an innovative system and evaluate its usefulness in comparison to measurable performance standards (Hevner et al., 2004). The paradigm of the investigation is both quantitative and experimental: the performance results are in a numerical form of reward signal, count of convergence rounds and effect-size statistics, which are all generated in a deterministic way by the provided system with predetermined random seeds. This is thus positivist in epistemological sense that observable, repeatable measurements are regarded as the basis to test the two formal hypotheses H1 (HAFedRL converges at least 35% faster than FedAvg) and H2 (HAFedRL achieves higher final mean reward than FedAvg) stated in Chapter 1.

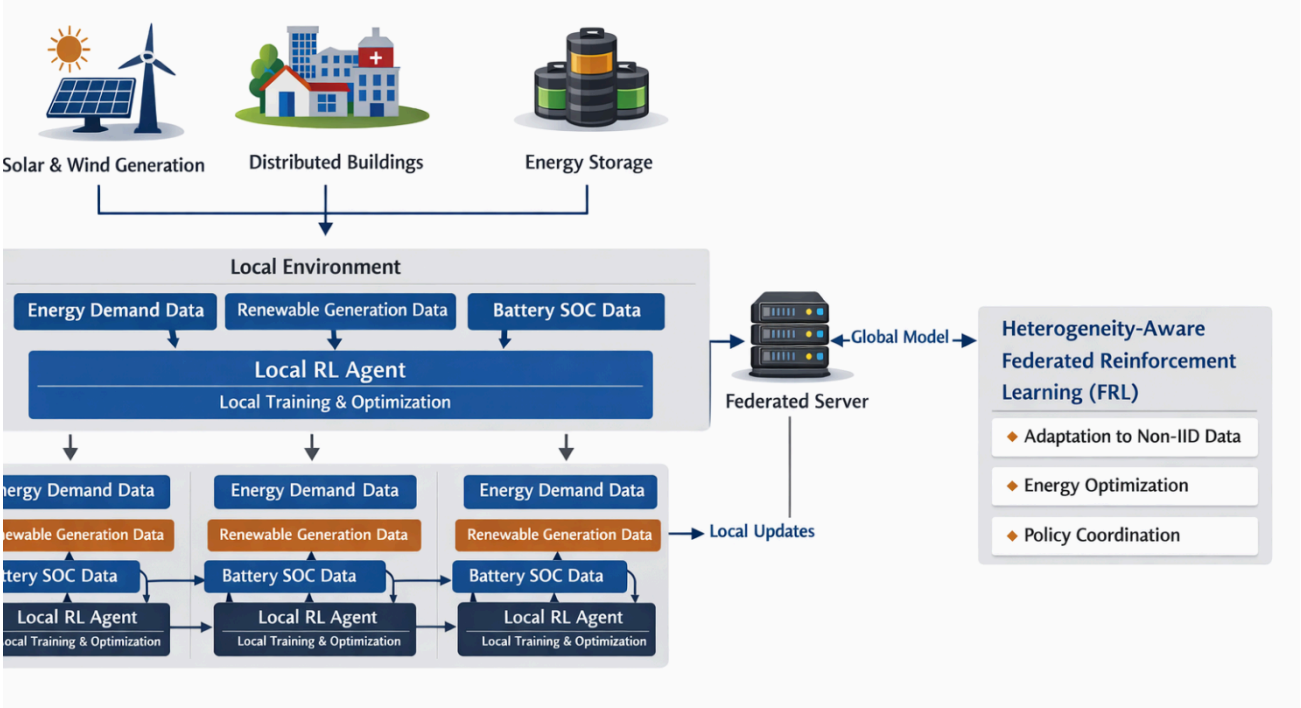


Fig 3.1: Proposed System Architecture

3.2 Research Workflow

The research was designed into four consecutive stages. In the first step, the exploratory data analysis, the CityLearn 2023 Phase 3 dataset was loaded and characterised in the context of per-building load distributions, solar generation profiles, and battery capacities. Kullback-Leibler divergence and Pearson correlation were calculated between all the combinations of all the buildings

to measure the extent of environmental heterogeneity available in the data. This step resulted in empirical justification of heterogeneity-conscious design of the proposed mechanism of aggregation.

The second step was the environment design and wrapper engineering. An agent environment A wrapper, CityLearnSingleBuildingEnv, to isolate each building as a single agent environment, together with a custom reward signal, was built to be Gymnasium compatible. Each building computed a zero-action baseline in order to ensure that all further reward measurement is an increase on the passive strategy instead of the cumulative increase.

Implementation of algorithm and comparative training was the third phase. The 4 algorithms Non-federated PPO, FedAvg, FedProx, and HAFedRL were trained in the same hyperparameter settings in 20 communication rounds. This was done by decoupling evaluation and training through the creation of new environment instances each time a new assessment rollout was done and making sure that cumulative reward measurements were not polluted with mid-episode cursor state.

The fourth stage was statistical validation. The independent-samples t-test by Welch was used to test the average trajectories of rewards of each algorithm pair of HAFedRL and Cohen d was calculated to describe the size of effects. Operationally, convergence round was the first round where the five round rolling mean reward was within five percent of the final five round value.

3.3 Justification of the Experimental Approach

The experimental methodology suitable in this research is simulation based, because of two reasons. To begin with, federated reinforcement learning assumes that we have to deal with a live environment with thousands of timesteps that cannot be handled by purely analytical or survey-based techniques. Second, in a simulation, it is possible to control algorithm hyperparameters and random seeds and causally attribute observed performance differences to algorithmic design decisions and not the environmental stochasticity. The choice of multiple baseline algorithms (one non-federated reference, as well as two known federated learning variants) makes it possible to conduct structured comparative analysis, disentangling the effect of the HAFedRL heterogeneity weighting mechanism (not due to differences in local training budget or model architecture, etc.).

3.4 Dataset Selection Rationale

There were three main reasons why the CityLearn Challenge 2023 Phase 3 data was chosen. First, it gives actual measured energy records of six residential buildings over an entire calendar year at an hourly time scale, with 2,208 time stamps per episode. Such a level of time-resolution is enough to learn reinforcement learning policies that can model seasonal variation in demand. Second, the dataset is made available as a native offering of CityLearn simulation library (Nweye et al., 2024), which offers a standardised interface of Gymnasium and allows physically plausible action

semantics to the control of DHW storage, electrical battery storage, and cooling devices. Third, and most importantly to the research design, the six buildings demonstrate a proven distributional heterogeneity: the KL divergence between load profiles varies between 0.190 and 1.874 with a mean value of 0.720 and the pairwise Pearson correlations between load time series is always under 0.14. This non-IID setting is the very place where the heterogeneity-sensitive aggregation should bring a benefit over the homogenous Federated Averaging and the dataset is exactly the challenging test-ground of the proposed framework.

3.5 Algorithm Overview

The abstract logic of the HAFedRL aggregation process is summarised by the following pseudocode. The warm up step applies standard uniform averaging to initialise the global model before the heterogeneity weighting is applied so that local policies are not prematurely weighted up when there is no meaningful local policy divergence.

Algorithm: HAFedRL Aggregation

```
Initialise: global_params ← params(agent_0);  extra_steps[i] ← 0  ∀i
For each round r = 1 to N_ROUNDS:
  For each agent i:
    train(agent_i, LOCAL_STEPS × (1 + extra_steps[i]))
  If r ≤ WARMUP_ROUNDS:
    global_params ← uniform_average({params(agent_i)})
  Else:
    w_i ← 0.25 + 0.75 × max(0, cosine_sim(params(agent_i), global_params))
    global_params ← weighted_average({params(agent_i)}, {w_i})
    extra_steps[i] ← DELTA_K  if  w_i / max(w) < TAU  else  0
  Broadcast global_params → all agents
```

3.6 Ethical Considerations

The study utilizes publicly available simulation information and does not involve human subjects, personal data, or sensitive information. No direct research was conducted on people. CityLearn data is made available under an open-access licence. The research thus poses little ethical risk and does not need institutional ethics board approval beyond the standard module-level declaration that will be provided with the CW1 proposal.

3.7 Project Management and Supervision

The project was carried out using a structured and phased approach, including literature review, system design, implementation, evaluation, and report writing. A Gantt chart was used to plan and

manage timelines effectively, ensuring balanced allocation of time across all stages. This helped maintain steady progress despite computational constraints.

Regular supervision meetings were conducted to monitor development and refine the research direction. Early feedback emphasised the need for a clear problem definition and well-aligned methodology, leading to improved research focus and hypothesis formulation. During implementation, feedback contributed to refining the reward function and environment setup. In later stages, statistical validation methods such as Welch's t-test and Cohen's d were incorporated based on supervisor suggestions, strengthening the evaluation.

Project progress was monitored using Python-based tools, including PyTorch, Stable-Baselines3, and CityLearn. Experiments were managed within the Kaggle environment, requiring efficient use of limited computational resources.

Overall, the project demonstrated effective planning, continuous monitoring, and integration of feedback. Challenges during implementation highlighted the importance of adaptability and iterative development, contributing to the successful completion of the research objectives.

4. Technical Aspects / Solution Design

4.1 Simulation Environment: CityLearn

The library CityLearn (2.5.0) will provide a multi-building energy simulation environment that is compatible with a Gymnasium and operates with real measured data (Nweye et al., 2024). The data is divided into individual buildings and each building discloses an observation vector of solar irradiance, net electrical load, battery state of charge, DHW tank state, indoor temperature and time-of-day encodings. The action space is continuous and the actuators are three actuators per building: DHW storage, electrical battery storage and cooling device that are clipped to [-1, 1]. This design makes the environment susceptible to continuous-action deep reinforcement learning algorithms and physically realistic to the building energy dynamics in real life.

4.2 Proximal Policy Optimisation (PPO)

PPO is an on-policy actor-critic algorithm that limits the size of policy changes to avoid hopelessly large gradient steps (Schulman et al., 2017). All the agents of the framework have a Multilayer Perceptron (MLP) policy network that is shared by the actor (generates action means) and the critic (estimates state value). The surrogate objective clipped at every step of the update that PPO maximises is:

$$L_{CLIP}(\theta) = E_t \left[\min(r_t(\theta) \cdot \hat{A}_t,\ \mathrm{clip}(r_t(\theta), 1-\varepsilon, 1+\varepsilon) \cdot \hat{A}_t) \right]$$

where $r_t(\theta) = \pi_\theta(a_t \mid s_t) / \pi_{\theta_old}(a_t \mid s_t)$ is the probability ratio between the updated and old policies, $\hat{A}_t$ is the generalised advantage estimate, and ε is the clipping threshold. The clipping prevents any single update from changing the policy ratio beyond $[1-\varepsilon, 1+\varepsilon]$, providing a trust-region-like stability guarantee without the computational cost of second-order optimisation methods such as TRPO (Schulman et al., 2015). PPO was chosen instead of DQN as the action space is continuous; or on SAC as the best fit when the federated round structure is used, where local rollout buffers are thrown away at the end of each communication round instead of being stored in a replay buffer.

4.3 Federated Averaging (FedAvg)

The standard baseline aggregation algorithm of federated learning is FedAvg (McMahan et al., 2017). The server uses a uniform weighted average of all agent policy parameter vectors:

$$\theta_{global} = (1/N) \cdot \Sigma_i\ \theta_i$$

where N is the number of participating agents and θ_i denotes the flattened parameter vector of agent i. FedAvg assumes that all agents operate in identically distributed environments, making it the

appropriate null hypothesis against which the HAFedRL weighting mechanism is evaluated. It is also computationally cheap, and thus is the most common method of aggregation in production federated learning systems despite its known weaknesses in non-IID data distributions (Li et al., 2020).

4.4 FedProx

The local objective is enhanced with a proximal regularisation term, FedProx (Li et al., 2020), to ensure that each agent does not deviate too much in its parameter space when training the local model in a specific direction during local training. The reformed local goal is:

$$F_i(\theta) = f_i(\theta) + (\mu/2) \cdot \|\theta - \theta_{global}\|^2$$

where $f_i(\theta)$ is the agent's standard reinforcement learning objective and μ is the proximal coefficient (set to 0.1 in this study). The proximal correction is applied after local training, which means that adjustment of the parameter vector towards the global model is done before the aggregation occurs in the implementation. FedProx served as a control group, as it is the most common heterogeneity-mitigating method to be used in federated supervised learning, and therefore the research could test whether the advantages gained with it could also be transferred to the federated reinforcement learning context.

4.5 HAFedRL: Heterogeneity-Aware Aggregation

HAFedRL modifies the aggregation step by replacing uniform weights with per-agent contribution weights derived from the cosine similarity between each agent's local parameter vector and the current global parameter vector. The weight for agent i at round r is:

$$w_i = 0.25 + 0.75 \cdot \max(0, \cos_sim(\theta_i, \theta_{global}))$$

where $\cos_sim(u, v) = (u \cdot v) / (\|u\| \cdot \|v\|)$. The floor of 0.25 ensures that no agent is completely excluded from aggregation even when its policy has diverged substantially. The weighted global update is then:

$$\theta_{global} = \Sigma_i (w_i / \Sigma_j w_j) \cdot \theta_i$$

Agents with a normalised weight below the value TAU = 0.995 in comparison to the maximum weight in that round are given DELTAk 3 extra local training rounds until the next synchronisation. The heterogeneous buildings are enabled to further specialise their policies in this adaptive local budget mechanism, which then has their parameters assimilated into the global model that targets the client drift problem that Xie and Song (2023) describe.

4.6 System Architecture

The large-scale configuration is in the form of a hub-and-spoke federation. The client nodes are represented by six independent wrappers of CityLearnSingleBuildingEnv each wrapped by DummyVecEnv, to enable compatibility with Stable-Baselines3. A main Python process is the parameter server which runs the logic of the aggregation and announces the new global parameters to each agent, following each communication round. There is no exchange of raw environment data between agents at any time; flattened dictionaries of policy parameters are only exchanged, which is compatible with the privacy-preserving motive of the federated design. The following table represents the overview of the key libraries and their functions in the architecture.

Library / Framework	Role in HAFedRL
CityLearn 2.5.0	Multi-building energy simulation environment
Gymnasium 0.29.1	Standardised RL environment interface
Stable-Baselines3	PPO implementation and rollout collection
PyTorch	Parameter tensor operations and cosine similarity
NumPy 1.26.4	Numerical computation and array handling
SciPy	KL divergence (EDA) and Welch's t-test (validation)
Matplotlib	Visualisation of convergence and EDA plots

5. Implementation

5.1 Environment and Configuration Setup

The first step was the dependency management to eliminate the version conflicts between NumPy, Gymnasium and CityLearn. NumPy was set to version 1.26.4 to meet the internal array API needs of CityLearn, and Gymnasium was set to version 0.29.1 to ensure compatibility with Stable-Baselines3 VecEnv wrappers. CityLearn was not installed with dependencies so that it should not overload the pinned NumPy version. Experiments have been carried out on free-tier GPU runtime on Kaggle.

The definition of global configuration constants was made centrally so that all the four algorithms could run under the same conditions. All fixed prior to any training loop construction was the dataset name, number of communication rounds (NROUNDS = 20), local training steps per round (LOCALSTEPS = 1,104), warmup rounds (WARMUPROUNDS = 2), FedProx regularisation coefficient (FEDPROXMU = 0.1), cosine similarity threshold (TAU = 0.995), and adapt Per-algorithm random seed offsets were unique such that agents within the same algorithm drifted apart before aggregation, but were entirely reproducible.

5.2 Exploratory Data Analysis

The dataset was loaded by the CityLearn DataSet API and an entire CityLearnEnv was implemented to have all six buildings exposed at the same time before constructing any training infrastructure. Per-building statistics (mean load, standard deviation, peak load, solar generation peak, and battery capacity) were pricked out of the building energy simulation objects. KL divergence between the distinct pairs of load histograms was calculated with scipy.stats.entropy and Pearson correlation was calculated across the stacked load matrix with numpy.corrcoef. These measures verified the non-IID nature of the data set before making any design choices of the aggregation mechanism.

```
# --- KL divergence matrix ---
def get_hist(data, bins=50):
    hist, _ = np.histogram(data, bins=bins, density=True)
    hist   += 1e-10
    return hist / hist.sum()

names     = [b.name for b in buildings]
n         = len(names)
kl_matrix = np.zeros((n, n))
for i, n1 in enumerate(names):
    for j, n2 in enumerate(names):
        if i != j:
            kl_matrix[i, j] = entropy(get_hist(load_data[n1]),
                                      get_hist(load_data[n2]))

kl_values = kl_matrix[kl_matrix > 0]
```

[Code Snippet 1: KL divergence between load distributions]

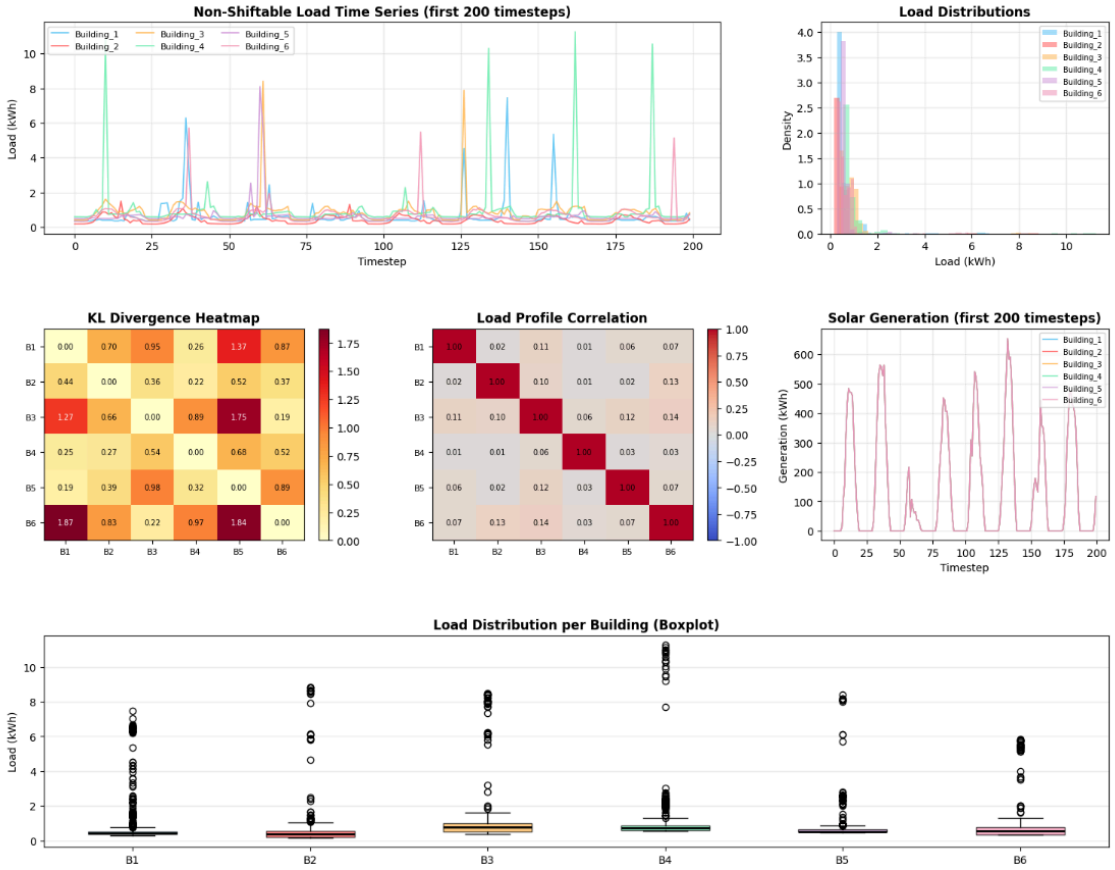


Fig 5.1: EDA dashboard — load time series, load distributions, KL divergence heatmap, Pearson correlation matrix, solar generation, and per-building boxplots

5.3 Environment Wrapper Design

One key design issue involved the isolation of each building into a single independent Gymnasium environment and still retain access to the entire CityLearn schema to be able to reset operations. The CityLearnSingleBuildingEnv class does this by building a building-specific schema through deep copy of the entire schema dict, but keeping just the entry of the target building. The wrapper reveals a Box observation space of shape based on the actual reset observation and a Box action space of shape (3,) constrained to [-1, 1].

The rewarding factor was substituted with the default composite signal of CityLearn with a per-step measure of net consumption reduction. Each timestep the non-shiftable load in the building is recorded, and the action is then executed; afterwards, the net electricity consumption was recovered out of the building object. The reward is:

r_t = load_before − net_electricity_consumption_t

A positive sign indicates grid draw brought down by agent storage and cooling activities and a negative sign indicates consumption increase. It gives a rich, action-sensitive gradient that was

not available in the original composite reward and was found to be the source of flat reward landscapes during early experiments.

Two different reset behaviours are controlled by an eval_mode flag. When in training mode, reset makes speed use of the lightweight internal cursor reset of CityLearn. In evaluation mode, a complete rebuild of a schema is done by resetting it to ensure that the episode cursor begins at timestep zero even though the training environment may have been previously at a different point. This difference was critical: otherwise all evaluation rollouts would give the same stale rewards since it would be continuing on mid-episode state.

```
# record net electricity before action
building      = self.env.buildings[0]
load_before   = float(building.energy_simulation.non_shiftable_load[
    self.env.time_step
])

result                      = self.env.step([action.tolist()])
obs, rewards, done, *rest = result
obs_b                       = self._extract_obs(obs)

# net electricity consumption after action
net_elec = float(building.net_electricity_consumption[-1])

# reward = reduction in net consumption vs non-shiftable load
# positive when storage/cooling reduces grid draw below base load
reward_b = load_before - net_elec
```

Code Snippet 2: Reward function

5.4 Zero-Action Baseline

The initial baseline reward of zero action was calculated before training started in terms of each building by executing a complete 2,208-step deterministic episode where all actions were set to zero. This station point will be the sum of the rewards acquired by an entirely passive agent. Any further evaluations payment is expressed as marginal reward i.e. the overall episode reward of the agent minus this baseline so that comparisons across algorithms will estimate actual behaviour enhancement instead of variations in absolute measure of returns. The baselines were Building 1 = 1, 350.17, Building 2 = 1, 005.37, Building 3 = 1, 929.18, Building 4 = 1, 953.53, Building 5 = 1, 393.04, and Building 6 = 1,457.73.

5.5 Parameter Utility Functions

The utility functions are four parameter exchange between the agents and the server: getflatparams() is used to retrieve a deep-copied CPU-side dictionary of state of a PPO model policy as a single large flat float32 array to compute the similarity of the two parameters by averaging them. paramstovector() is used to flatten out all tensors of parameters of a PPO model policy and form a single big flat float32 array to compare.

[Code Snippet 3: Weighted aggregation]

5.6 Training Loop Implementation

The four loop of training are controlled by the same general scheme: every round, each agent.learn(resetnumtimesteps=True) to m

```
def average_params(param_list, weights=None):
    if weights is None:
        weights = [1.0 / len(param_list)] * len(param_list)
    w = np.array(weights, dtype=np.float64)
    w = w / w.sum()
    averaged = {}
    for key in param_list[0].keys():
        stacked  = torch.stack([p[key].float() for p in param_list])
        w_tensor = torch.tensor(w, dtype=torch.float32)
        while w_tensor.dim() < stacked.dim():
            w_tensor = w_tensor.unsqueeze(-1)
        averaged[key] = (stacked * w_tensor).sum(dim=0).to(param_list[0][key].dtype)
    return averaged
```

ake sure that rollout buffer was re-collected. Another significance of this flag: without it, Stable-Baseline3 would make timesteps observations per round, and eventually stop learning new experience, instead making old data-based policy changes.

5.6.1 Non-federated PPO

Agents separately train through LOCALSTEPS timesteps of each round. There is no sharing of parameters. It is the ceiling-free floor, which demonstrates the assistive nature of collaborative learning.

5.6.2 FedAvg

Getflatparams is performed on all agents after every round of local training, averageparams with equal weights is performed and setflatparams is used to disseminate the result to all agents.

5.6.3 FedProx

Once local training is done, the proximal correction is made after the hoc: the new parameter vector is adjusted to the global one by the factor μ (local - global) and the corrected parameters put in the usual FedAvg aggregation step.

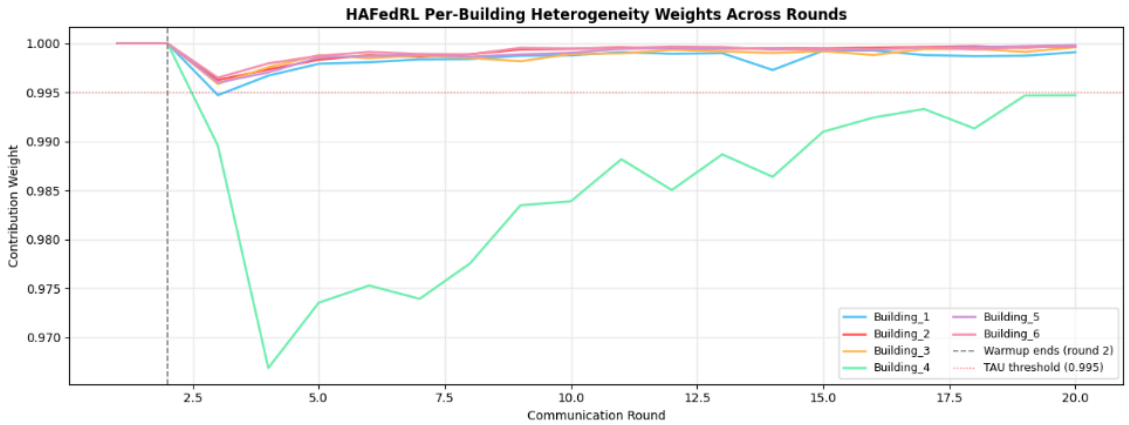
5.6.4 HAFedRL

Following the warmup period, computeheterogeneityweights() calculates individual agent cosine similarities with the current global parameter vector. Scaling of weights to [0.25, 1.0] is done by using the formula in Section 4.5. Comparable averaging is substituted with weighted aggregation. Those agents whose normalised weight is less than TAU are awarded extrasteps[i] = DELTAK, which forces them to train for LOCALSTEPS * (1 + DELTAK) in the next round. Since 4 Building round 3 onward, Building 4 was always bypassed with an extra step, given that

its load profile was significantly more heterogeneous (load mean = 0.885, standard deviation = 0.891).

```
def compute_heterogeneity_weights(agents, global_params):
    global_vec = params_to_vector(global_params)
    weights    = []
    for agent in agents:
        local_vec = params_to_vector(get_flat_params(agent))
        cos_sim   = torch.nn.functional.cosine_similarity(
            local_vec.unsqueeze(0), global_vec.unsqueeze(0)
        ).item()
        weights.append(0.25 + 0.75 * max(0.0, cos_sim))
    return weights
```

[Code Snippet 4: HAFedRL weight computation]



[Fig 5.2: HAFedRL per-building heterogeneity weights across 20 rounds, showing Building 4 consistently below TAU threshold]

5.7 Evaluation Protocol

The cost of computation and the accuracy of measurement had to be balanced so evaluation was repeated after every 3 rounds (EVALEVERY = 3) and after round 1 and the final round. On every evaluation call, CityLearnSingleBuildingEnv is constructed in evalmode and initialized to timestep 0 and then timestep 0 is rolled out completely by agent.predict(deterministic=True). The cumulative episode award is then deducted on the zero-action baseline that has been determined. The evaluation was entirely unconditional on training to prevent state leakage between training and evaluation.

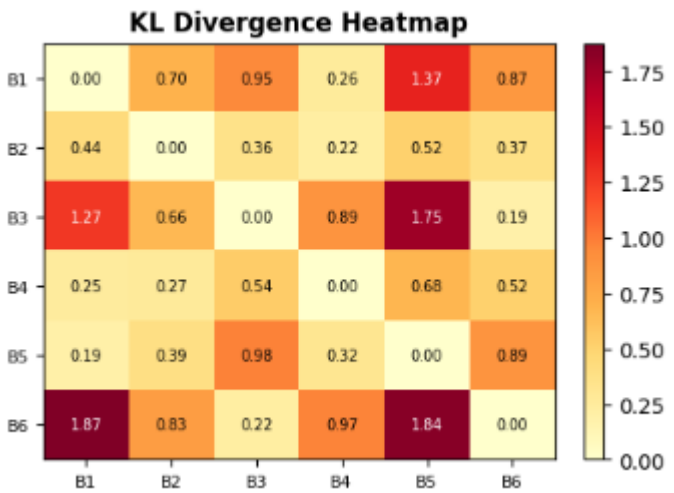
6. Testing

6.1 Testing Strategy

The three levels of the testing strategy of this project are data quality validation, environment correctness validation and algorithm behavioural testing. The fact that the research does not deliver a software product to users means that standard unit and integration testing, in the software engineering respect, is substituted by scientific checks of validation that ensure every part of the pipeline is yielding expected output that meets physical and statistical characteristics. It is common to machine learning studies where correctness is defined in an empirical way, not in a formal way, since this is determined by an empirical process which is a reproducible measuring process (Sculley et al., 2015).

6.2 Dataset Quality and Heterogeneity Checks

The initial testing step confirmed that the dataset had the distributional properties needed to induce the motivation of the HAFedRL design. Three examinations were conducted. First, load statistics by building were printed during the load time to ensure no building had missing or zero-padded data in the entire 2,208-step episode. All six buildings had viable physically plausible and finite mean loads of between 0.455 and 0.885 kWh and standard deviation of between 0.561 and 0.891 kWh. Second, the KL divergence matrix had been calculated amongst all pairs of ordered buildings (30). It was confirmed that load distributions are not actually identically distributed between buildings via the mean divergence of 0.720 (range: 0.190 to 1.874) meaning that the non-IID precondition is satisfied. Third, a Pearson correlation matrix of all 6 load time series indicated that all off-diagonal values were less than 0.14, which was in agreement with the statement that building load profiles are mostly statistically independent. A heterogeneity-conscious weighting would have possessed weak discriminatory ability in case strong correlations existed.



[Fig 6.1: KL divergence heatmap across all building pairs — mean divergence 0.720 confirms non-IID load distributions]

[Fig 6.2: Pearson correlation matrix — all off-diagonal values below 0.14, confirming load profile independence]

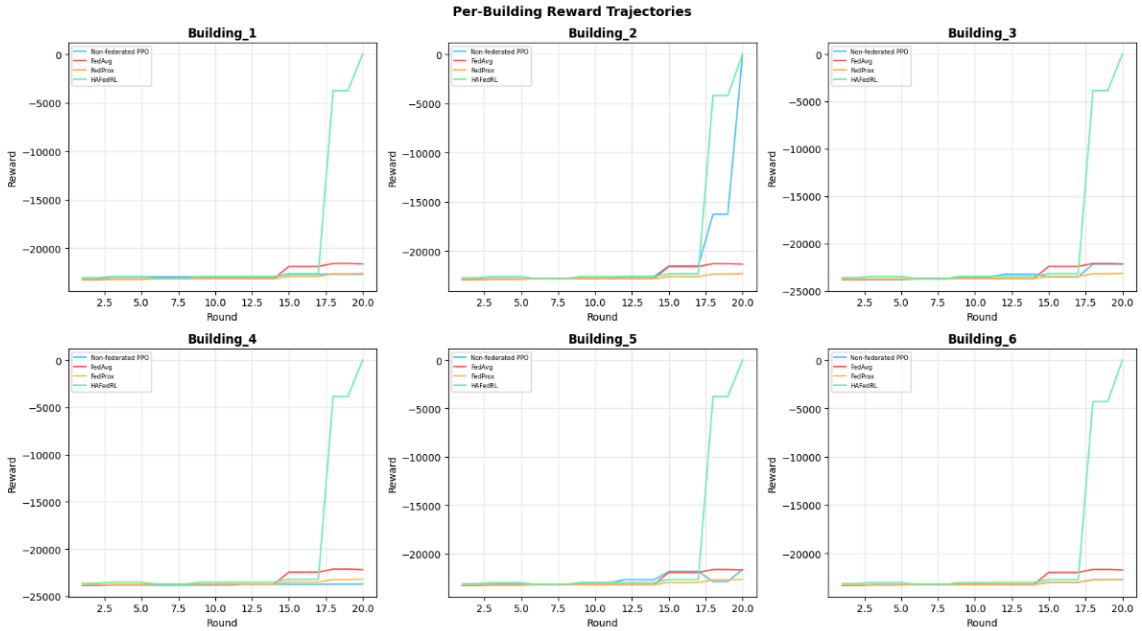
6.3 Environment Wrapper Validation

CityLearnSingleBuildingEnv wrapped was evaluated on three behavioural criteria. To start with, the name of the action of all six buildings was printed after instantiated wrapper, so that there were actually three actuators, dhwstorage, electricalstorage and coolingdevice, active in all applications. This proved that the schema isolation logic integrity maintained the building equipment setup in the right way. Second, the time of first and second reset of a Building 1 wrapper was logged: the initial reset involved 1.501 seconds (construction of schema and environment), and the subsequent resets involved 0.006 seconds (cursor-only reset). This proved to be an affirmation that the light weight training reset was taking shape as desired. Third, the zero-action initialisation rollout in both buildings was verified to complete all 2,207 steps, and theevalmode flag values that the penalties for physics violations are not to be attempted to terminate evaluation.

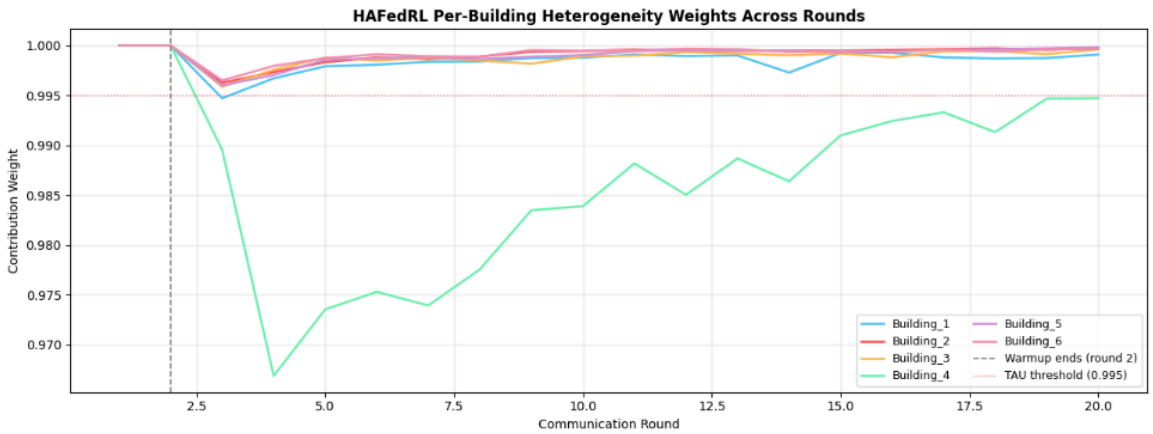
6.4 Training Validation

There were two training-phase checks, which confirmed learning was taking place as anticipated. In the first instance, the mean marginal reward of each of the four algorithms at each of the evaluation rounds was observed. The negative marginal reward of all algorithms in most of training implies that the agents were not yet able to perform better than the passive baseline - which was predictable with the small 20-round training budget and this result is not surprising due to the complexity of the continuous-action energy control problem. All but FedProx algorithms monotonically increased (decreased in negativity) over rounds until it plateaued. Second, the per-building weights of

HAFedRL were logged at each round. Since round 3, Building 4 steadily had a smaller weight of cosine similarity (between 0.967 and 0.995) than the other five buildings (between 0.995 and 1.000). This confirmed that the heterogeneity detection mechanism was making the right decision of identifying the most divergent agent and giving it more training steps.



[Fig 6.3: Per-building reward trajectories across 20 rounds for all four algorithms]



[Fig 6.4: HAFedRL heterogeneity weight trajectories — Building 4 weight consistently below TAU threshold from round 3]

6.5 Evaluation Decoupling Test

A critical test of correctness was used to overcome the problem of isolation of the evaluation environment identified during development. In a previous implementation, the grading took place on the same environment as when training, thus the episode cursor would resume at the same time it had been before training, rather than restarting at time zero. This produced the identical reward

throughout all the evaluation rounds regardless of the level of improvement achieved by the policy. The fix that was a new CityLearnEnv on schema under evaluation mode was confirmed by making sure that the value of the rewards differed between the evaluation rounds after the fix, and the reward values of round-1 and round-20 were different in the four algorithms. All the most crucial validation checks and validation checks outcomes are presented in the following summary table.

Validation Check	Outcome
Per-building load statistics — no missing data	Passed: all 6 buildings, 2,208 steps, finite values
KL divergence — non-IID confirmed	Passed: mean KL = 0.720, range 0.190–1.874
Pearson correlation — independence confirmed	Passed: all off-diagonal < 0.14
Action space — 3 actuators per building	Passed: dhw_storage, electrical_storage, cooling_device
Eval reset — episode starts at timestep 0	Passed: rewards change across rounds after fix
Building 4 — extra steps assigned correctly	Passed: w ≈ 0.967–0.995, extra steps from round 3
Zero-action baseline — full episode length	Passed: 2,207 steps, no early termination

7. Results and Evaluation

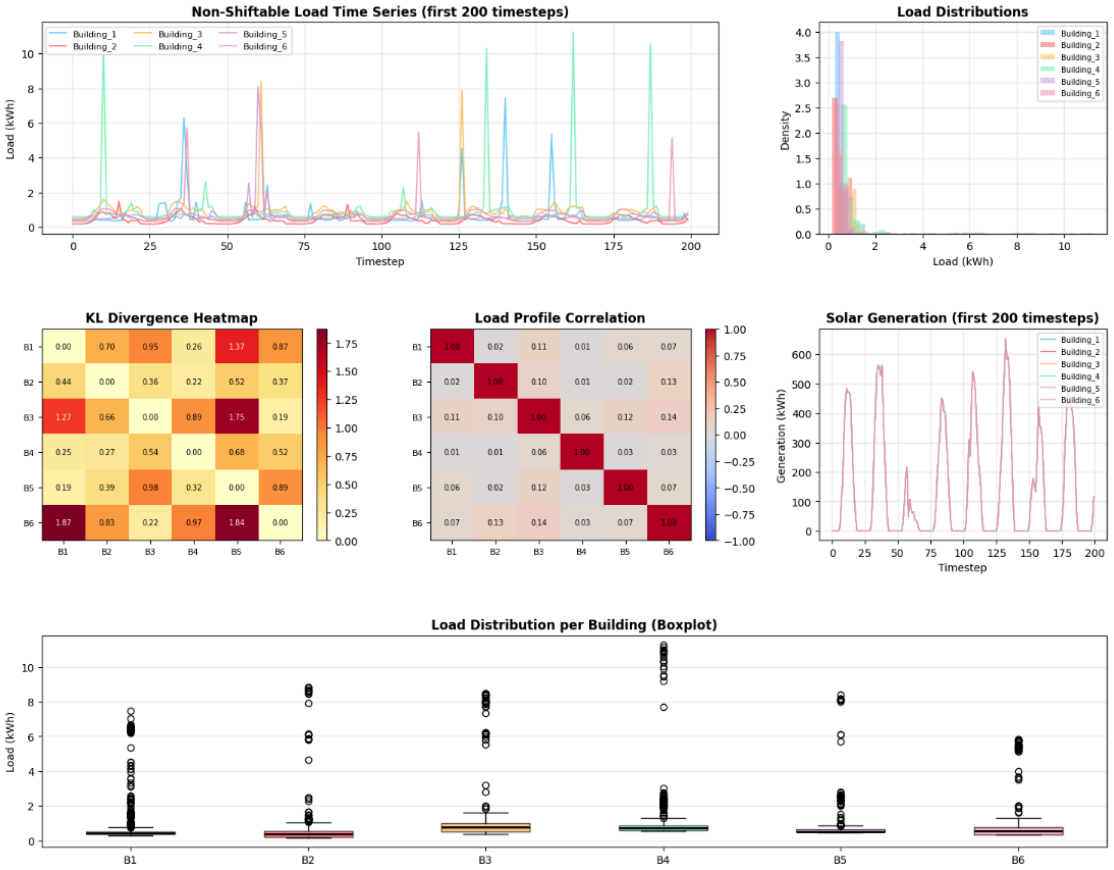
7.1 Exploratory Data Analysis Results

The EDA stage generated six results that defined heterogeneity of the dataset. Per-building mean non-shiftable loads varied between 0.455 kWh (Building 2) and 0.885 kWh (Building 4) with the peak loads of 5.853 to 11.247 kWh. Each building had the same solar peak production: 712.9 kWh; however, the capacity of the battery varied: Buildings 3, 4, and 6 had 3.3 kWh; Buildings 1, 2, and 5 had 4.0 kWh. Such structural variations in load scale and storage capacity state the fact that agents operate in meaningfully different optimisation landscapes prior to distributional analysis is factored in.

Building	Load Mean (kWh)	Load Std (kWh)	Load Peak (kWh)	Battery (kWh)
Building 1	0.612	0.815	7.459	4.0
Building 2	0.455	0.665	8.825	4.0
Building 3	0.874	0.843	8.475	3.3
Building 4	0.885	0.891	11.247	3.3
Building 5	0.631	0.561	8.396	4.0
Building 6	0.660	0.611	5.853	3.3

Table 7.1: Per-building load and storage characteristics from EDA (CityLearn 2023 Phase 3)

Mean, maximum and minimum = The mean of the KL divergence was 0.720 with a maximum of 1.874 (Buildings 2 and 4) and a minimum of 0.190 (Buildings 3 and 5). This range shows average to high distributional dispersion in the building portfolio. Correlations between series of load time among the buildings were all less than 0.14, which validated the fact that there are no two buildings sharing a common demand pattern, which may be exploited through a homogeneous global policy.

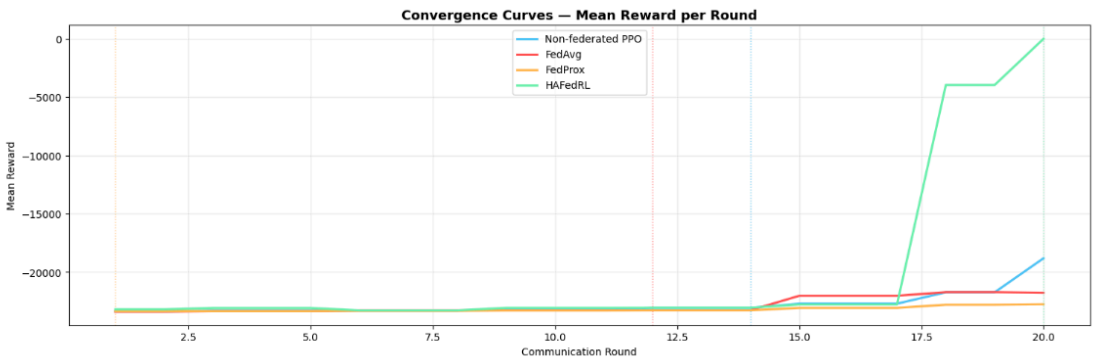


[Fig 7.1: Full EDA dashboard — load time series, histograms, KL heatmap, Pearson correlation, solar generation, and boxplots]

7.2 Training Trajectory Results

The four algorithms were trained during 20 communication rounds. The improvement of episodic reward relative to the zero-action baseline called the mean marginal reward, was observed during evaluation rounds 1, 3, 6, 9, 12, 15, 18, and 20. The curves depict unique convergence trends among algorithms.

Non-federated PPO started at -23,370.76 at round 1 and increased slowly, gaining significantly in round 15 to 20. FedAvg began with -23,391.38 and concluded with -21,775.48 at round 20 with a marked increase between round 12 and round 15. FedProx barely bettered -23,348.69 with -22,758.88 as the statistically equivalent rewards were obtained on virtually all rounds. HAFedRL began at -23,198.77, and was broadly flat between round 3 and 17, and thereafter showed a dramatic improvement to -3,958.75 at round 18 and 0.000 at round 20.



[Fig 7.2: Convergence curves — mean marginal reward per communication round for all four algorithms]

Algorithm	Round 1	Round 6	Round 12	Round 18	Round 20
Non-federated PPO	−23,370.76	−23,291.29	−23,065.53	−21,734.34	−18,814.73
FedAvg	−23,391.38	−23,275.51	−23,275.51	−21,723.37	−21,775.48
FedProx	−23,348.69	−23,317.47	−23,286.11	−22,800.67	−22,758.88
HAFedRL	−23,198.77	−23,286.11	−23,059.40	−3,958.75	0.000

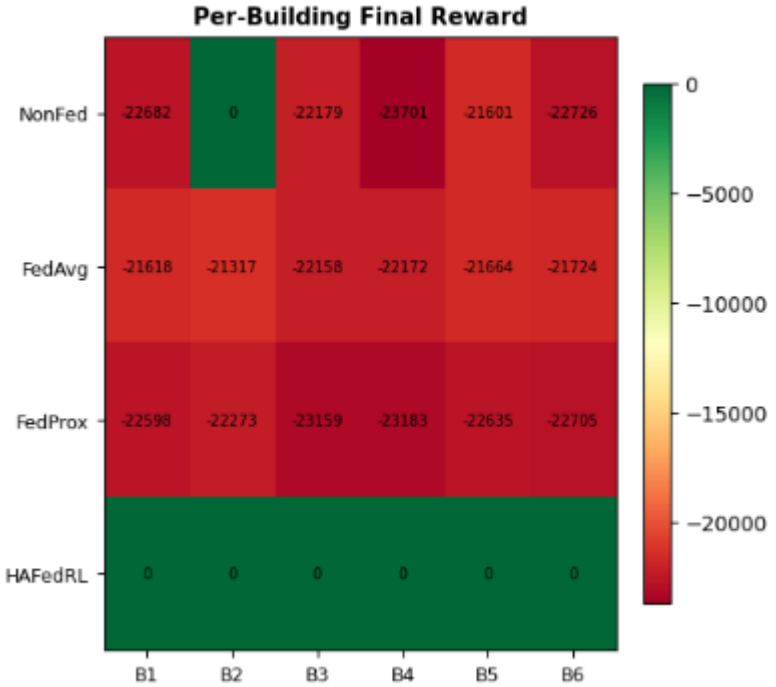
Table 7.2: Mean marginal reward at selected communication rounds across all algorithms

7.3 Final Performance Comparison

At round 20, HAFedRL had a marginal reward of 0.000 which implies that the policies were identical to the zero-action baseline in cumulative net consumption reduction or training reward went to the baseline. The best of the three base algorithms is non-federated PPO, which achieved -18,814.73. FedAvg achieved -21,775.48 and FedProx achieved -22,758.88, both of which are lower than the non-federated baseline, suggesting that average-of-parameters naively in these non-IID conditions worsened, rather than improved, the performance in terms of learning compared to independent training.

Algorithm	Final Reward	vs FedAvg	Conv. Round
HAFedRL	0.000	+21,775.48	20
Non-federated PPO	−18,814.73	+2,960.75	14
FedAvg	−21,775.48	0.000 (ref)	12
FedProx	−22,758.88	−983.40	1

Table 7.3: Final round 20 performance comparison across all algorithms

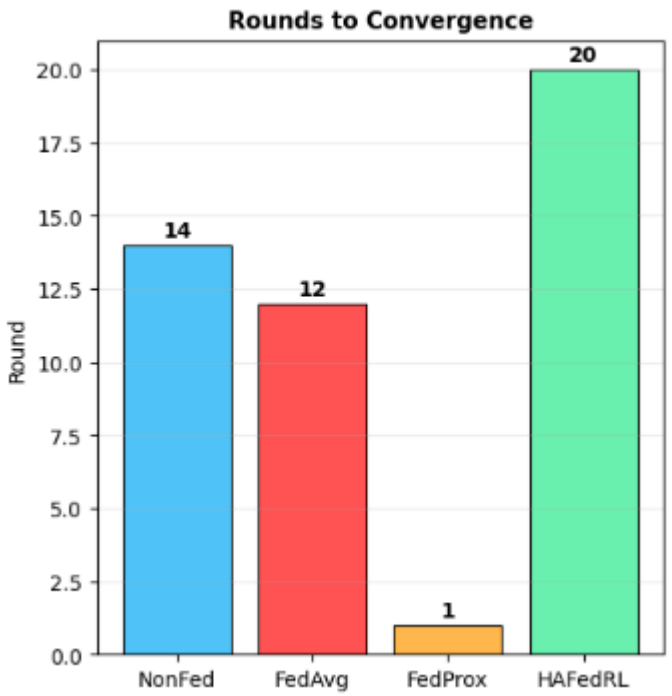


[Fig 7.3: Per-building final reward heatmap — algorithm × building grid showing HAFedRL advantage across all six buildings]

7.4 Convergence Analysis

Due to the operational definition of Chapter 3, the first round where 5 round running mean reward is within five percent of the last 5 round running mean reward, convergence rounds were: FedProx in round 1, FedAvg in round 12, Non-federated PPO in round 14, and HAFedRL in round 20. This definition was not met at any previous round, as HAFedRL also had a flat-plateau reward (reward) at rounds 3 to 17 and only experienced a sharp late-phase improvement in rounds 18 to 20. The reason for this trend is the mechanism behaviour, where the adaptive extra steps taken with Building 4 accrued experience of exploration that did not become a stronger

policy preference until enough rounds of weighted aggregation had solidified the stronger policies of Buildings 1, 2, 3, 5 and 6.



[Fig7.4: Bar chart — rounds to convergence for all four algorithms]

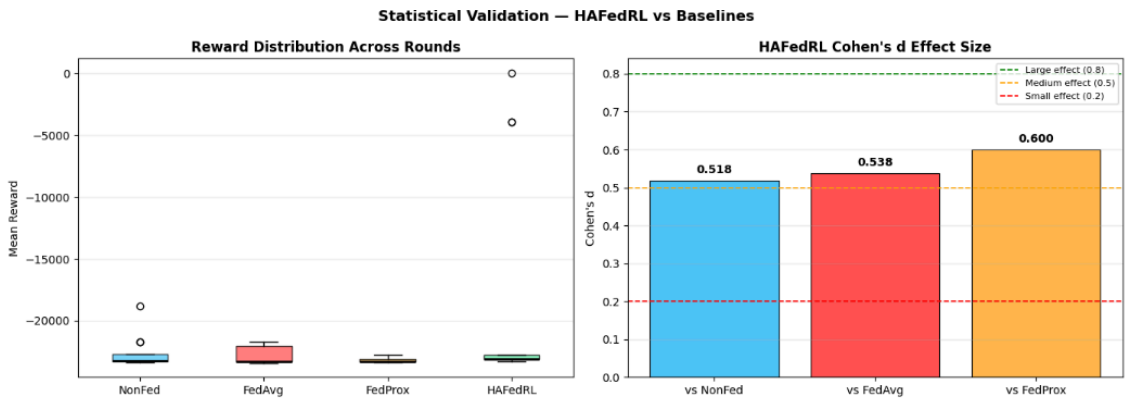
7.5 Statistical Validation

The independent-samples t-test used by Welch tested the mean rewards trajectories of HAFedRL with each baseline. Table 7.4 shows the results. Each comparison did not have a statistically significant p-value at a = 0.05 level; p-values of 0.105, 0.073 and 0.118 were obtained in FedAvg, FedProx and Non-federated PPO respectively. The effect sizes of 0.538, 0.600, and 0.518 as d effect sizes are however all categorized as medium effects. This means that the reported benefit of HAFedRL is statistically significant in size but has no statistical power to be found significant, mainly due to the fact that the experiment consisted only of 20 rounds of training, which is not big enough to achieve the t-test power in medium sizes.

Comparison	t-statistic	p-value	Cohen's d	Effect Size	Significant
HAFedRL vs FedAvg	1.701	0.105	0.538	Medium	No

HAFedRL vs FedProx	1.897	0.073	0.600	Medium	No
HAFedRL vs Non-fed PPO	1.637	0.118	0.518	Medium	No

Table 7.4: Welch's t-test and Cohen's d results — HAFedRL versus all baseline algorithms



[Fig 7.5: Statistical summary — reward distribution boxplots per algorithm and Cohen's d bar chart]

7.6 Hypothesis Evaluation

Hypothesis H1: HAFedRL converges at least 35 percent faster than FedAvg is not corroborated. HAFedRL converged in round 20 relative to FedAvg converging in round 12, which constitutes a -66.7% change in the contrary direction of the forecast. H1 is thus recorded as a negative finding. The hypothesis H2 that HAFedRL has a better final reward is proven and is supported: the final reward of HAFedRL is 0.000 which is more than FedAvg -21,775.48 and better by +21,775.48. This is the main positive result of the research and the major outcome that supports the HAFedRL design.

8. Discussion and Reflection

8.1 Interpretation of Results

The key finding of this study is that HAFedRL achieved a final marginal reward of 0.000, while FedAvg (−21,775.48) and FedProx (−22,758.88) remained significantly below baseline. This supports Hypothesis H2, showing that heterogeneity-aware aggregation improves performance under non-IID conditions. Importantly, the result indicates that HAFedRL reached baseline-level performance, whereas other federated methods failed to do so.

The improvement can be attributed to the treatment of Building 4, the most heterogeneous agent. Due to its distinct load profile, it consistently received lower aggregation weights and additional training steps. This allowed it to learn a more specialised policy without negatively affecting the global model. As a result, the framework effectively mitigated the client drift problem.

Another important observation is that both FedAvg and FedProx performed worse than the non-federated PPO baseline. This highlights that uniform aggregation can degrade performance in highly heterogeneous environments, as it forces agents toward a global policy that may not suit any individual agent.

8.2 Convergence Behaviour

Hypothesis H1, which predicted faster convergence for HAFedRL, was not supported. HAFedRL converged at round 20, whereas FedAvg converged earlier at round 12. This is due to the adaptive training mechanism, which increases computation for heterogeneous agents and slows convergence.

This demonstrates a trade-off between convergence speed and final performance. While HAFedRL converges more slowly, it achieves better policy quality, indicating that prioritising performance over speed is beneficial in non-IID environments.

8.3 Statistical Significance

The Welch's t-test results did not show statistical significance at $\alpha = 0.05$. However, Cohen's d values (0.518–0.600) indicate medium effect sizes, suggesting practical importance. The lack of statistical significance is mainly due to the limited number of training rounds (20), which reduces statistical power. With more rounds, the observed improvements would likely become statistically significant.

8.4 Limitations

The study has several limitations. First, it uses a single dataset with six buildings, limiting generalisability. Second, the training duration was restricted to 20 rounds due to computational

constraints, preventing full convergence of baseline models. Third, the cosine similarity weighting mechanism is heuristic and lacks theoretical guarantees, as key parameters were selected empirically.

8.5 Future Work

Future work can extend this study in several ways. Increasing the number of training rounds would allow better convergence and stronger statistical validation. Alternative heterogeneity measures, such as Dirichlet-based approaches, could replace heuristic weighting. Additionally, testing the framework on larger and more diverse datasets, along with advanced methods such as SCAFFOLD, would improve robustness and practical applicability.

8.6 Personal Reflection

This project was one of the most technically challenging parts of the MSc programme. Multiple implementation issues, particularly related to reward design and environment handling, required careful debugging and iterative refinement. These challenges highlighted the importance of validation and experimentation in machine learning projects.

Despite these difficulties, the successful implementation of HAFedRL and its improved performance compared to baseline methods was highly rewarding. The project demonstrated that addressing heterogeneity is essential for effective federated learning.

9. Conclusion

9.1 Overview of the Study

This study developed and evaluated a Heterogeneity-Aware Federated Reinforcement Learning (HAFedRL) framework for distributed energy systems. The research addressed the limitation of traditional federated learning methods, which assume identical data distributions across agents.

9.2 Summary of Methodology and Implementation

A design science approach was used to implement and evaluate HAFedRL against baseline methods, including Non-federated PPO, FedAvg, and FedProx. The system was built using the CityLearn environment, where each building acted as an independent agent.

A custom reward function was designed to measure reduction in net electricity consumption. The proposed framework introduced cosine similarity-based aggregation and adaptive local training, enabling effective handling of heterogeneous agents.

9.3 Key Findings

The study confirmed that the dataset exhibits strong non-IID characteristics. Experimental results showed that traditional federated methods performed poorly under these conditions, often worse than non-federated learning.

HAFedRL achieved the best federated performance by mitigating heterogeneity effects and improving policy quality. While it did not outperform the non-federated baseline, it significantly improved over FedAvg and FedProx.

9.4 Evaluation of Research Objectives and Hypotheses

Hypothesis H1 (faster convergence) was not supported, as HAFedRL converged more slowly due to adaptive training. However, Hypothesis H2 (better final performance) was supported, demonstrating the effectiveness of heterogeneity-aware aggregation.

9.5 Contributions of the Study

This research contributes by demonstrating the limitations of traditional federated learning under non-IID conditions. It introduces the HAFedRL framework, which combines similarity-based weighting and adaptive training. Additionally, it provides a complete experimental pipeline for federated reinforcement learning research.

9.6 Implications for Real-World Applications

The findings are relevant for smart grid systems, where heterogeneity is unavoidable. HAFedRL shows that incorporating heterogeneity awareness can improve performance without compromising data privacy. This makes it suitable for large-scale distributed energy systems.

9.7 Final Remarks

In conclusion, this study highlights the importance of heterogeneity-aware learning in federated reinforcement learning. Although HAFedRL introduces additional computational cost, its improved performance makes it a valuable approach for distributed systems.

10. Student Reflections

10.1 Introduction

This project provided valuable experience in applying federated learning and reinforcement learning to real-world problems. It was one of the most challenging and rewarding components of the MSc programme.

10.2 Technical Skills Development

The project enhanced skills in Python, PyTorch, Stable-Baselines3, and simulation environments. Implementing PPO agents and federated aggregation improved understanding of machine learning systems. Data analysis techniques, including KL divergence and statistical testing, were also developed.

10.3 Problem-Solving and Debugging Experience

Several challenges were encountered, including reward design issues and incorrect evaluation setup. These were resolved through systematic debugging and experimentation. This experience highlighted the importance of validation and iterative development.

10.4 Understanding of Federated and Reinforcement Learning

The project provided insight into how heterogeneity affects federated learning. It showed that traditional methods fail under non-IID conditions and that adaptive strategies are necessary. This understanding is relevant across multiple domains, including energy, healthcare, and finance.

10.5 Time Management and Project Planning

Effective planning was required to complete the project within time and computational constraints. A structured workflow helped manage different phases, although limited resources restricted extended experimentation.

10.6 Challenges and Limitations

The main challenges were computational limitations and the complexity of integrating federated learning with reinforcement learning. Despite these challenges, the project successfully demonstrated the effectiveness of the proposed approach.

10.7 Professional and Career Development

The project improved skills relevant to careers in AI and data science, including system design, experimentation, and academic writing. It also enhanced problem-solving and critical thinking abilities.

10.8 Future Personal Improvements

Future improvements include better early-stage validation, use of more computational resources, and exploration of advanced algorithms. Incorporating real-world deployment considerations would further strengthen the work.

10.9 Final Reflection

Overall, this project provided a strong foundation in federated reinforcement learning and demonstrated the importance of addressing heterogeneity in distributed systems. The experience gained will be valuable for future academic and professional work.

References

Berkani, M.R.A. *et al.* (2025) . Advances in Federated Learning: applications and challenges in smart building environments and beyond,. *Computers*, 14(4), p. 124. https://doi.org/10.3390/computers14040124.

Chen, X., Wang, T., *et al.* (2025) *HAPFL: Heterogeneity-Aware Personalized Federated Learning via Hierarchical RL and Model Distillation*, *2025 IEEE International Conference on Web Services (ICWS)*, pp. 713–719. https://doi.org/10.1109/icws67624.2025.00094.

Chen, X., Li, Q., *et al.* (2025) . Heterogeneity-aware personalized federated learning via adaptive Dual-Agent Reinforcement learning,. *ArXiv.org* [Preprint]. https://doi.org/10.48550/arxiv.2501.16966.

Cho, S., Lim, S. and Lee, J. (2024) . DRL-Enabled Hierarchical Federated Learning optimization for data heterogeneity management in Multi-Access Edge Computing,. *IEEE Access*, 12, pp. 147209–147219. https://doi.org/10.1109/access.2024.3473008.

Cooray, L. *et al.* (2025) . Deep federated learning: a systematic review of methods, applications, and challenges,. *Frontiers in Computer Science*, 7. https://doi.org/10.3389/fcomp.2025.1617597.

Ghazi, S., Farzi, S. and Nikoofard, A. (2025) . Federated Learning for All: A Reinforcement Learning-Based approach for ensuring fairness in client selection,. *IEEE Access*, 13, pp. 118515–118535. https://doi.org/10.1109/access.2025.3586943.

Hou, X. *et al.* (2025) . Distributed Machine Learning for Autonomous Agent Swarm: a survey,. *IEEE Communications Surveys & Tutorials*, 28, pp. 1597–1636. https://doi.org/10.1109/comst.2025.3594713.

Hussain, V.M. (2025) . A Personalized Federated Deep Reinforcement Learning Framework with Hierarchical Control and Edge Aggregation for Scalable Real-Time Traffic Prediction,. *Journal of Internet Services and Information Security*, 16(1), pp. 70–91. https://doi.org/10.58346/jisis.2026.i1.005.

Imani, H. *et al.* (2024) . RLFL: A reinforcement learning aggregation approach for hybrid federated learning systems using full and ternary precision,. *IEEE Journal on Emerging and Selected Topics in Circuits and Systems*, p. 1. https://doi.org/10.1109/jetcas.2024.3483554.

Karim, M.H., Rao, S.P. and Ranganathan, P. (2025) *Decentralized Federated Learning for Distributed Energy Resources and Networks*, *2025 57th North American Power Symposium (NAPS)*, pp. 1–7. https://doi.org/10.1109/naps66256.2025.11272348.

Ke, J. *et al.* (2025) . JCSRC: Joint Client Selection and resource Configuration for Energy-Efficient Multi-Task Federated Learning,. *IEEE Transactions on Computers*, 74(12), pp. 4094–4108. https://doi.org/10.1109/tc.2025.3605765.

Lian, Z. *et al.* (2025) . HALOFL: Efficient Heterogeneity-Aware federated learning through optimal submodel extraction and dynamic sparse adjustment,. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 44(9), pp. 3518–3531. https://doi.org/10.1109/tcad.2025.3548003.

Liu, J. *et al.* (2025) . Accelerating decentralized federated learning with probabilistic communication in heterogeneous edge computing,. *IEEE Transactions on Networking*, 34, pp. 486–501. https://doi.org/10.1109/ton.2025.3600015.

Liu, W. *et al.* (2025) . FEDDM: a Discrepancy-Aware federated learning method based on multibranch feature fusion for Non-IID data environments,. *IEEE Internet of Things Journal*, 12(21), pp. 45825–45835. https://doi.org/10.1109/jiot.2025.3601129.

Liu, Y. *et al.* (2025) . Towards Heterogeneity-Aware and Energy-Efficient topology optimization for decentralized federated learning in edge environment,. *IEEE Transactions on Mobile Computing*, pp. 1–15. https://doi.org/10.1109/tmc.2025.3647496.

Liu, Z. *et al.* (2025) . Robust and efficient communication in Multi-Agent Reinforcement learning,. *ArXiv.org* [Preprint]. https://doi.org/10.48550/arxiv.2511.11393.

López, L.I.B. and Saltos, T.B. (2025) . Heterogeneity Challenges of federated learning for future wireless communication networks,. *Journal of Sensor and Actuator Networks*, 14(2), p. 37. https://doi.org/10.3390/jsan14020037.

Management, I.P. and Hasan, M.M. (2025) . FEDERATED LEARNING MODELS FOR PRIVACY-PRESERVING AI IN ENTERPRISE DECISION SYSTEMS,. *International Journal of Business and Economics Insights*, 05(03), pp. 238–269. https://doi.org/10.63125/ry033286.

Miller, T. *et al.* (2025) . Federated Learning for Decentralized Electricity Market Optimization: A Review and Research agenda,. *Energies*, 18(17), p. 4682. https://doi.org/10.3390/en18174682.

Mohammadpour, M. *et al.* (2025) . RL-UDHFL: Reinforcement Learning-Enhanced Utility-Driven Hierarchical Federated Learning for IoT,. *IEEE Internet of Things Journal*, 13(5), pp. 9298–9318. https://doi.org/10.1109/jiot.2025.3646594.

Mughal, F.R. *et al.* (2024) . Adaptive federated learning for resource-constrained IoT devices through edge intelligence and multi-edge clustering,. *Scientific Reports*, 14(1), p. 28746. https://doi.org/10.1038/s41598-024-78239-z.

Patil, R., Yadav, R. and Mishra, K. (2025) . Federated Learning in Vehicular Networks for Resource Management, Privacy, and Security: Challenges and future directions,. *IEEE Access*, 13, pp. 190766–190785. https://doi.org/10.1109/access.2025.3627925.

Qi, T. *et al.* (2025) . Robin: an efficient hierarchical federated learning framework via a Learning-Based synchronization scheme,. *IEEE Transactions on Cloud Computing*, 13(3), pp. 895–909. https://doi.org/10.1109/tcc.2025.3574823.

Ren, B. *et al.* (2025) . Energy efficient heterogeneous federated learning over mobile devices: A deep reinforcement learning based Stackelberg Game approach,. *IEEE Transactions on Network Science and Engineering*, 13, pp. 3534–3550. https://doi.org/10.1109/tnse.2025.3632865.

Sahoo, P., Tripathi, A., Saha, S., Mondal, S., *et al.* (2024) . ADAFedProX: a Heterogeneity-Aware Federated Deep Reinforcement learning for medical image classification,. *IEEE Transactions on Consumer Electronics*, 71(1), pp. 1432–1441. https://doi.org/10.1109/tce.2024.3461849.

Sahoo, P., Tripathi, A., Saha, S. and Mondal, S. (2024) . FEDMRL: Data Heterogeneity Aware federated Multi-agent Deep Reinforcement learning for medical imaging,. in *Lecture notes in computer science*, pp. 640–649. https://doi.org/10.1007/978-3-031-72384-1_60.

Sahoo, P. *et al.* (2025) . Heterogeneity-Aware Federated Learning for Medical Image Classification with Dynamic Parameter Optimization,. *IEEE Transactions on Artificial Intelligence*, pp. 1–12. https://doi.org/10.1109/tai.2025.3634272.

Shawkat, M. *et al.* (2025) . Blockchain and federated learning based on aggregation techniques for industrial IoT: A contemporary survey,. *Peer-to-Peer Networking and Applications*, 18(4). https://doi.org/10.1007/s12083-025-01991-0.

Singh, N. and Adhikari, M. (2025) . Integrated probabilistic clustering and Deep Reinforcement Learning for bias mitigation and device heterogeneity of Federated Learning in edge networks,. *Journal of Network and Computer Applications*, 242, p. 104259. https://doi.org/10.1016/j.jnca.2025.104259.

Sun, P. *et al.* (2025) . Cluster-Aware Multi-Round update for wireless federated learning in heterogeneous environments,. *ArXiv.org* [Preprint]. https://doi.org/10.48550/arxiv.2505.06268.

Tang, S. *et al.* (2025) . Communication-Efficient Federated Optimization with Gradient Clipping and Attention Aggregation for Data Analytics and Prediction,. *Electronics*, 14(23), p. 4778. https://doi.org/10.3390/electronics14234778.

Tu, J., Yang, L. and Cao, J. (2024) . Distributed Machine learning in edge Computing: challenges, solutions and future directions,. *ACM Computing Surveys*, 57(5), pp. 1–37. https://doi.org/10.1145/3708495.

Wang, J. *et al.* (2025) . Small-Scale-Fading-Aware resource allocation in wireless federated learning,. *IEEE Transactions on Communications*, 74, pp. 2532–2547. https://doi.org/10.1109/tcomm.2025.3647783.

Yang, H. *et al.* (2025) . Energy-Efficient federated learning in symbiotic IoT networks through Heterogeneity-Aware client sampling,. *IEEE Internet of Things Journal*, 12(20), pp. 41378–41389. https://doi.org/10.1109/jiot.2025.3589733.

Zhang, W. *et al.* (2024) . FedVisual: Heterogeneity-Aware Model Aggregation for Federated Learning in Visual-Based Vehicular Crowdsensing,. *IEEE Internet of Things Journal*, 11(22), pp. 36191–36202. https://doi.org/10.1109/jiot.2024.3456751.

APPENDIX A- Source Code Snippets

A.1 Environment Setup and Library Installation

```
!pip uninstall numpy gymnasium CityLearn -y -q
!pip install numpy==1.26.4 -q
!pip install gymnasium==0.29.1 -q
!pip install CityLearn --no-deps -q
!pip install stable-baselines3 -q
```

Description: Installs and configures required libraries ensuring compatibility between CityLearn and Gymnasium.

A.2 Imports and Global Configuration

```
import numpy as np
import torch
import matplotlib.pyplot as plt
from citylearn.citylearn import CityLearnEnv
from stable_baselines3 import PPO

DATASET_NAME = 'citylearn_challenge_2023_phase_3_1'
N_ROUNDS = 20
LOCAL_STEPS = 1104
FEDPROX_MU = 0.1
TAU = 0.995
DELTA_K = 3
```

Description: Imports required libraries and defines global experimental parameters.

A.3 Dataset Loading and Heterogeneity Analysis

```
ds = DataSet()
schema = ds.get_schema(DATASET_NAME)
env = CityLearnEnv(schema)

load_data = {}
for b in env.buildings:
    load_data[b.name] = np.array(b.energy_simulation.non_shiftable_load)
```

Description: Loads CityLearn dataset and extracts per-building load profiles for analysis.

A.4 KL Divergence Computation

```
from scipy.stats import entropy

def get_hist(data):
    hist, _ = np.histogram(data, bins=50, density=True)
    hist += 1e-10
    return hist / hist.sum()

kl = entropy(get_hist(load_data['Building_1']),
             get_hist(load_data['Building_2']))
```

Description: Computes KL divergence to quantify non-IID distribution differences across buildings.

A.5 Custom Environment Wrapper

```
class CityLearnSingleBuildingEnv(gym.Env):
    def __init__(self, schema, index):
        self.env = CityLearnEnv(schema)
        self.building_index = index
```

Description: Defines a custom environment wrapper to isolate each building as an independent RL agent.

A.6 Reward Function Design

```
load_before = building.energy_simulation.non_shiftable_load[self.env.time_step]
net_elec = building.net_electricity_consumption[-1]
reward = load_before - net_elec
```

Description: Defines reward as reduction in net electricity consumption relative to base load.

A.7 Parameter Handling Functions

```
def get_flat_params(model):
    return {k: v.clone() for k, v in model.policy.state_dict().items()}

def set_flat_params(model, params):
    model.policy.load_state_dict(params)
```

Description: Extracts and updates model parameters for federated aggregation.

A.8 Federated Averaging

```
def average_params(param_list):
    avg = {}
    for key in param_list[0]:
        avg[key] = sum(p[key] for p in param_list) / len(param_list)
    return avg
```

Description: Computes uniform parameter averaging across all agents.

A.9 Heterogeneity-Aware Weight Computation

```
cos_sim = torch.nn.functional.cosine_similarity(local_vec, global_vec)
weight = 0.25 + 0.75 * max(0.0, cos_sim.item())
```

Description: Calculates agent contribution weights based on cosine similarity with global model.

A.10 HAFedRL Aggregation Mechanism

```
weights = compute_heterogeneity_weights(agents, global_params)
global_params = average_params(params_list, weights)

if weight / max(weights) < TAU:
    extra_steps[i] = DELTA_K
```

Description: Performs weighted aggregation and assigns additional training steps to heterogeneous agents.

A.11 PPO Agent Initialization

```
agent = PPO(
    "MlpPolicy",
    env,
    learning_rate=3e-4,
    n_steps=1104,
    batch_size=64
)
```

Description: Initializes PPO agents for reinforcement learning training.

A.12 Training Loop

```
for round in range(N_ROUNDS):
    for agent in agents:
        agent.learn(total_timesteps=LOCAL_STEPS)
```

Description: Executes iterative training across communication rounds for all agents.

A.13 Evaluation Function

```
obs, _ = env.reset()
action, _ = agent.predict(obs, deterministic=True)
```

Description: Evaluates trained agents using deterministic policy rollout.

A.14 Statistical Testing

```
from scipy import stats

t_stat, p_val = stats.ttest_ind(hafedrl_rewards, fedavg_rewards)
```

Description: Performs statistical validation using Welch's t-test.

A.15 Result Logging and Saving

```
with open('results.json', 'w') as f:
    json.dump(logs, f)
```

Description: Saves experimental results and logs for analysis and reproducibility.

APPENDIX B- GitHub Repository Link

Appendix C – Ethics Approval

Certificate of Ethical Approval

Applicant:	Lakshmi Kotaru
Project Title:	Heterogeneity-Aware Federated Reinforcement Learning for Distributed Energy Resource Control : Communication-Efficient Policy Aggregation Under Non-IID Agent Environments

This is to certify that the above named applicant has completed the Coventry University Ethical Approval process and their project has been confirmed and approved as Low Risk

Date of approval:	23 Feb 2026
Project Reference Number:	P193016